Algorithms for the Linear Ordering Problem: Implementation Exercise #1

Daniele Cioffi
Heuristic Optimization

April 2026

1 Abstract

I implemented and compared 14 local search configurations for the *Linear Ordering Problem* (LOP), combining three neighborhood structures (Transpose, Exchange, Insert), two pivoting rules (First Improvement, Best Improvement), two initialization strategies (Random, Chenery–Watanabe), and two Variable Neighborhood Descent (VND) variants. I tested the algorithms on a benchmark of 78 instances of sizes $n = 150$ and $n = 250$. Results are evaluated calculating the Average Percentage Deviation (APD) from the best-known value provided in the project. The statistical significance of the results is calculated using paired t-tests and Wilcoxon signed-rank tests. Transpose is the weakest neighborhood (mean APD $> 15\%$), while Exchange, Insert, and VND reach mean APD below 4%. The best configurations — First+Insert+Chenery and both VND variants — achieve mean APD around 1.7% and are not significantly different from each other. The Chenery–Watanabe initialization consistently outperforms random initialization, and First Improvement matches or surpasses Best Improvement for the Insert neighborhood while running several times faster.

2 Approach

The LOP is a combinatorial optimization problem and since the problem becomes computationally expensive as size grows, exact approaches are impractical for larger instances.

I assessed the following design choices:

1. **Neighborhood structure** — how the set of neighbors of a solution is defined;
2. **Pivoting rule** — whether to accept the *first* improving neighbor or to scan the full neighborhood and accept the *best*;
3. **Initial solution** — whether to start from a random permutation or from the Chenery–Watanabe (CW) greedy construction heuristic.

In addition, two Variable Neighborhood Descent (VND) strategies that cycle through all three neighborhoods are evaluated.

All algorithms are implemented in C and tested on a benchmark set of 78 instances. The remainder of this report is organized as follows. Section 3 formalizes the LOP. Section 4 describes the implementation. Section 5 presents experimental results and statistical analysis. Section 6 concludes.

3 The LOP - Linear Ordering Problem

Given a $n \times n$ matrix $C = (c_{ij})$ of non-negative real weights, the LOP outputs a permutation π of $\{1, \ldots, n\}$ that maximizes

$$f(\pi) \;=\; \sum_{i=1}^{n} \sum_{j=i+1}^{n} c_{\pi_i, \pi_j}. \tag{1}$$

The goal is to find an ordering of the rows (and columns) of C such that the sum of entries in the upper triangle of the permuted matrix is maximized.

Benchmark. The benchmark is composed of 78 instances: 39 with $n = 150$ and 39 with $n = 250$. For each instance the Average Percentage Deviation (APD) is computed:

$$\text{APD} \;=\; 100 \times \frac{f^* - f(\hat{\pi})}{f^*}, \tag{2}$$

where f^* is the best-known value and $f(\hat{\pi})$ is the value found by the algorithm. A lower APD indicates better solution quality.

4 Implementation and algorithms

4.1 Implementation

The solver is implemented in C and organized into five modules: argument parsing and main loop (`main.c`), instance loading (`instance.c`), all search algorithms (`optimization.c`), a random number generator (`utilities.c`), and timing (`timer.c`).

The RNG seed is set as the sum of all entries in the cost matrix.

4.2 Neighborhoods

I tested three neighborhood structures commonly used for LOP search procedures.

Transpose ($\mathcal{N}_T$). Swap two *adjacent* elements at positions i and $i+1$. The neighborhood size is $n-1$. The incremental cost change for transposing elements π_i and π_{i+1} is $\Delta = c_{\pi_{i+1},\pi_i} - c_{\pi_i,\pi_{i+1}}$, computed in $O(1)$.

Exchange ($\mathcal{N}_E$). Swap any two elements at positions i and j ($i < j$). The neighborhood size is $\binom{n}{2}$. The incremental cost change (`deltaExchange`) is computed in $O(n)$.

Insert ($\mathcal{N}_I$). Remove the element at position i and reinsert it at position $j \neq i$. The neighborhood size is $n(n-1)$. The incremental cost change (`deltaInsert`) is computed in $O(n)$.

4.3 Pivoting Rules

First Improvement. Iterate over neighbors in a fixed order; accept the first move with $\Delta > 0$ and restart the scan from the beginning. Terminate when no improving neighbor is found.

Best Improvement. Scan the entire neighborhood; accept the move with the largest positive Δ. Terminate when no improving neighbor exists.

Both pivoting rules use function pointers so that any neighborhood can be passed as a callback without code duplication.

4.4 Initial Solutions

Random. Generate a uniformly random permutation of $\{1, \ldots, n\}$.

Chenery–Watanabe (CW). Compute the row sum $a_i = \sum_{j=1}^{n} c_{ij}$ for each element and sort elements in decreasing order of a_i. The rationale is that an element with a large row sum contributes more when placed early in the permutation, so ranking by a_i yields a greedy approximation to the optimum.

4.5 Variable Neighborhood Descent

Two VND configurations are tested, both initialized with the Chenery–Watanabe solution and using First Improvement as the internal pivoting rule:

- **VND-TEI**: neighborhoods cycled in order $\mathcal{N}_T \to \mathcal{N}_E \to \mathcal{N}_I$.
- **VND-TIE**: neighborhoods cycled in order $\mathcal{N}_T \to \mathcal{N}_I \to \mathcal{N}_E$.

The VND procedure optimizes the current solution with the first neighborhood; upon reaching a local optimum, it moves to the next. Any improvement in a non-first neighborhood resets control to $\mathcal{N}_T$. The procedure terminates when the solution is a local optimum for all three neighborhoods simultaneously.

4.6 Experimental Setup

All 14 configurations are run on each of the 78 instances. Computation time is measured as wall-clock time per run. Statistical comparisons use paired tests (each algorithm pair is compared instance-by-instance) at significance level $\alpha = 0.05$: a two-sided paired t-test and a Wilcoxon signed-rank test (with normal approximation). I considered the Wilcoxon as primary since APD values may not be normally distributed.

5 Results

5.1 Summary Statistics

Table 1 reports, for each of the 14 configurations, the mean APD, the standard deviation of APD, and the total wall-clock time summed over all 78 instances. A lower mean APD indicates better solution quality.

Table 1: Summary of results over 78 instances. $\bar{\delta}$: mean APD (%); σ_δ: standard deviation of APD (%); Time: total wall-clock time (s) across all instances.

Pivoting	**Neighborhood**	**Init**	$\bar{\delta}$	σ_δ	**Time (s)**
First	Transpose	Random	34.61	3.80	0.10
First	Transpose	Chenery	15.86	1.42	0.44
First	Exchange	Random	2.82	0.48	346.08
First	Exchange	Chenery	2.41	0.44	287.50
First	Insert	Random	2.02	0.45	295.33
First	Insert	Chenery	1.69	0.36	217.23
Best	Transpose	Random	34.47	3.80	0.09
Best	Transpose	Chenery	15.73	1.43	0.42
Best	Exchange	Random	3.60	0.50	53.61
Best	Exchange	Chenery	2.96	0.43	47.75
Best	Insert	Random	2.30	0.46	58.52
Best	Insert	Chenery	1.84	0.32	51.51
VND-TEI	—	Chenery	1.78	0.38	143.26
VND-TIE	—	Chenery	1.74	0.40	122.64

The results reveal a clear two-tier structure. **Transpose** stands alone as the worst neighborhood by a large margin: mean APD of $\approx 35\%$ with random initialization and $\approx 16\%$ with CW. All other configurations — Exchange, Insert, and VND — form a second tier with mean APD between 1.7% and 3.6%.

Within the second tier, **Insert** outperforms Exchange for any given pivoting rule and initialization pairing. The best single configuration is **First+Insert+Chenery** (mean APD = 1.69%), closely followed by the two VND variants ($\approx$ 1.74–1.78%).

Regarding initialization, the CW heuristic consistently produces lower APD than random initialization for every neighborhood and pivoting rule. The benefit is most pronounced for Transpose but is significant across the board.

Regarding the pivoting rule, Best Improvement does not outperform First Improvement for the Insert neighborhood: **First+Insert** yields lower mean APD than **Best+Insert** in both initialization variants, while running 4–5$\times$ faster. For Exchange, the relationship reverses: Best Improvement achieves lower APD than First Improvement, though at a substantially shorter computation time for Best ($\approx$ 50 s vs. $\approx$ 290–346 s).

5.2 Statistical Tests

All 91 pairwise comparisons were performed. Table 2 summarizes the most informative comparisons.

Table 2: Selected pairwise Wilcoxon signed-rank tests ($\alpha = 0.05$). p_W: Wilcoxon p-value; Sig.: statistically significant difference.

Algorithm A	**Algorithm B**	p_W	**Sig.**
Effect of neighborhood (First Improvement, Random init)			
First+Transpose+Rand	First+Exchange+Rand	< 0.0001	Yes
First+Transpose+Rand	First+Insert+Rand	< 0.0001	Yes
First+Exchange+Rand	First+Insert+Rand	< 0.0001	Yes
Effect of initialization			
First+Transpose+Rand	First+Transpose+CW	< 0.0001	Yes
First+Exchange+Rand	First+Exchange+CW	< 0.0001	Yes
First+Insert+Rand	First+Insert+CW	< 0.0001	Yes
Effect of pivoting rule			
First+Exchange+Rand	Best+Exchange+Rand	< 0.0001	Yes
First+Exchange+CW	Best+Exchange+CW	< 0.0001	Yes
First+Insert+Rand	Best+Insert+Rand	< 0.0001	Yes
First+Insert+CW	Best+Insert+CW	0.0007	Yes
Top-tier comparisons			
First+Insert+CW	VND-TEI+CW	0.0710	No
First+Insert+CW	VND-TIE+CW	0.2887	No
Best+Insert+CW	VND-TEI+CW	0.0866	No
Best+Insert+CW	VND-TIE+CW	0.0117	Yes
VND-TEI+CW	VND-TIE+CW	0.3491	No
Notable non-significant comparison			
First+Exchange+CW	Best+Insert+Rand	0.2865	No

Neighborhood effect. All three pairwise comparisons between distinct neighborhoods are highly significant ($p_W < 0.0001$). Transpose is significantly worse than both Exchange and Insert; Exchange is significantly worse than Insert.

Initialization effect. The Chenery–Watanabe initialization provides a statistically significant improvement over random initialization for all three neighborhoods ($p_W < 0.0001$ in every case).

Pivoting rule effect. Contrary to intuition, First Improvement yields *better* solutions than Best Improvement for the Insert neighborhood (significantly so in both initialization settings). This indicates that the specific path through the search space explored by First Improvement leads to higher-quality local optima for this problem. For the Exchange neighborhood, Best Improvement is significantly better than First Improvement but requires much less computation time (≈ 50 s vs. ≈ 290–346 s), because the Best Improvement scan terminates after a smaller number of improving moves.

Top-tier analysis. The four best-performing configurations — First+Insert+CW (1.69%), VND-TIE (1.74%), VND-TEI (1.78%), and Best+Insert+CW (1.84%) — are largely indistinguishable statistically. First+Insert+CW is not significantly different from either VND variant ($p_W = 0.07$ and 0.29), and Best+Insert+CW is not significantly different from VND-TEI ($p_W = 0.09$). Only the pair Best+Insert+CW vs. VND-TIE reaches significance ($p_W = 0.012$) in favor of VND-TIE. The two VND variants are equivalent ($p_W = 0.35$). A notable boundary case is that First+Exchange+CW (2.41%) is not significantly different from Best+Insert+Rand

(2.30%, $p_W = 0.29$), showing that a good initialization can compensate for a weaker neighborhood when the pivoting rule is chosen appropriately.

5.3 Computational Cost

The time figures in Table 1 reveal that solution quality and computation time are closely linked. Transpose algorithms are extremely fast (< 0.5 s total) but produce poor solutions. First Improvement with Exchange requires the most time overall ($\approx$ 290–346 s), because it performs many small improving moves; Best Improvement with Exchange converges faster ($\approx$ 48–54 s) and achieves better quality. For Insert, First Improvement is faster than Best Improvement ($\approx$ 217–295 s vs. $\approx$ 52–59 s) and also achieves better quality — making First+Insert+CW the dominant choice among single-neighborhood algorithms. VND-TIE (122 s) is the most time-efficient top-tier configuration.

6 Conclusion

This study compared 14 local search configurations for the LOP on a benchmark of 78 instances. The main conclusions are:

1. **Transpose is substantially inferior.** Both First and Best Improvement with the Transpose neighborhood yield mean APD of 15–35%, far above all other configurations. Transpose's restricted neighborhood (only adjacent swaps) appears fundamentally insufficient to escape poor local optima.
2. **Exchange, Insert, and VND are all competitive.** All configurations using Exchange, Insert, or VND achieve mean APD below 4%. The best configurations (First+Insert+Chenery, VND-TEI, VND-TIE) reach $\approx 1.7\%$ and are not significantly different from each other.
3. **Initialization consistently matters.** The Chenery–Watanabe construction significantly improves solution quality for every neighborhood–pivoting combination tested ($p_W < 0.0001$ in all cases).
4. **First Improvement beats Best Improvement for Insert.** For the Insert neighborhood, First Improvement reaches significantly better local optima than Best Improvement while also being faster. For Exchange the reverse holds, but with Best Improvement converging more quickly.
5. **VND achieves top-tier quality efficiently.** VND-TIE reaches mean APD of 1.74% in 123 s total, matching First+Insert+Chenery in quality while cycling through all three neighborhoods. The ordering of neighborhoods within VND (TEI vs. TIE) has no statistically significant effect.

The recommended configuration therefore is **First Improvement with the Insert neighborhood and Chenery–Watanabe initialization**, which achieves the lowest mean APD (1.69%) among all single-algorithm variants with a reasonable total runtime. When a shorter runtime is preferred one can use **VND-TIE**.